CSE-2340 · SOFTWARE DEVELOPMENT 1

Week 6 · Classes 1 and 2

The DOM

Selectors, content, attributes, classList, createElement and appendChild

Md Sadman Hafiz

Lecturer, Dept. of CSE, IIUC

Autumn 2026

C1 · 0-5 MIN

Recall and today's outcome

RECALL

From Class 3

- What do map, filter and find each do?
- What is the last index of an array of 4 items?
- Why did your function return undefined?
- What shape was the students data?

OUTCOME

By 50 minutes you can

- Find any element on a page with a CSS selector
- Read and change text, attributes and classes
- Build new elements in JavaScript and add them
- Turn an array of objects into a page

The two lines from last class get explained today

- document.querySelector("#results-body").innerHTML = rows;
- You wrote them, they worked, and nobody told you why. That ends now.

Not today: making things happen when the user clicks. That is Class 3. Today the script runs once, when the page loads.

C1 · 5-15 MIN · CONCEPT

The DOM is not your HTML file

1

You write

index.html, a text file sitting on disk

2

The browser reads it

and builds a live tree of objects in memory

3

That tree is the DOM

JavaScript changes the tree, and the page changes with it

What this means in practice

- Your HTML file never changes
- Reload the page and everything resets
- The DOM only lives while the tab is open

See it for yourself

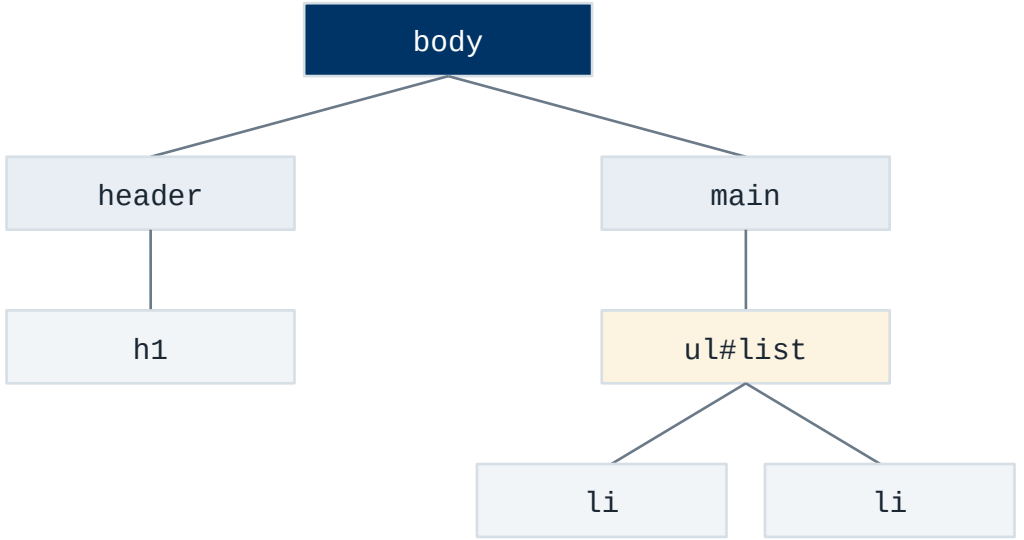
- F12, then the Elements tab
- That is the DOM, not your file
- Right-click an element, Edit as HTML

The one sentence

HTML is the instruction. The DOM is what the browser built from it. JavaScript edits what was built.

A page is a tree

```
<body>
  <header>
    <h1>My Page</h1>
  </header>
  <main>
    <ul id="list">
      <li>One</li>
      <li>Two</li>
    </ul>
  </main>
</body>
```



The words you will hear

- Parent, child, sibling — the same family words
- ul is the parent of both li elements
- The two li elements are siblings

Why a tree matters

- You add a child to a parent, never to the page
- appendChild always needs somewhere to put it

Finding an element: you already know the syntax

```
document.querySelector("#list")       // id,     first match
document.querySelector(".card")       // class,  first match
document.querySelector("h1")          // tag,    first match
document.querySelector("nav a")       // any CSS selector works

document.querySelectorAll(".card")    // ALL matches, as a NodeList

document.getElementById("list")       // older, id only, no #
```

The good news

- It takes a CSS selector
- You learned those in Week 3
- Nothing new to memorise

No match? You get null

querySelector never throws for a bad selector. It quietly returns null, and the error arrives one line later.

Use querySelector. getElementById is on the slide so you recognise it online.

REMEMBER

The # and the dot

- querySelector("#list") — with the hash
- getElementById("list") — without it
- Forgetting the # is the most common DOM bug of all

querySelectorAll gives you a list, not an element

```
const cards = document.querySelectorAll(".card");

cards.length            // 3
cards[0]                // the first element
cards.textContent = "x" // DOES NOTHING. Silent.

// Change all of them: loop
for (const c of cards) {
  c.textContent = "Hello";
}

cards.forEach(c => c.classList.add("done"));   // also fine
```

The rule

- Singular name, singular result: querySelector gives one element
- Plural name, plural result: querySelectorAll gives a list you must loop

TOP MISTAKE

A list has no textContent

- No error appears
- Nothing on the page changes
- You stare at it for ten minutes

A NodeList is almost an array

- It has length, indexes and forEach
- It has no map, filter or find
- Need those? Array.from(cards)

Reading and changing content

```
const title = document.querySelector("#title");

title.textContent               // read it
title.textContent = "New title" // replace it

// textContent treats everything as plain text
title.textContent = "<b>Hi</b>";   // shows:  <b>Hi</b>

// innerHTML parses it as HTML
title.innerHTML  = "<b>Hi</b>";    // shows:  Hi  (in bold)
```

Use textContent

- For anything that is just words
- Faster, and cannot break your page

The symptom you will see

- Tags showing literally on the page means textContent where you wanted innerHTML
- Your layout silently breaking means innerHTML where you wanted textContent

Use innerHTML carefully

- Only when you are building real HTML
- Never with text a user typed
- A visitor can type a tag and run code on your page

Attributes and properties

```
const photo = document.querySelector("#photo");

photo.setAttribute("src", "images/new.jpg");
photo.getAttribute("alt");
photo.removeAttribute("hidden");

// Common ones have a direct shortcut
photo.src  = "images/new.jpg";
link.href  = "https://iiuc.ac.bd";
input.value = "Rahim";          // not an attribute. A property.
```

Which should I use?

- The shortcut, when there is one
- setAttribute for anything custom

Your own data on an element

- <li data-id="C241002"> → li.dataset.id gives "C241002"
- This is how you remember which item a row belongs to

TRAP

input.value

- getAttribute("value") gives what the HTML said
- input.value gives what the user typed
- You almost always want .value

classList: change how something looks

```
const box = document.querySelector("#box");

box.classList.add("highlight");
box.classList.remove("hidden");
box.classList.toggle("open");        // on if off, off if on
box.classList.contains("open");      // true / false

box.classList.add("a", "b");         // several at once


// Avoid this
box.style.backgroundColor = "yellow";
box.style.border = "2px solid orange";

// Prefer this
box.classList.add("highlight");
```

Keep CSS in the CSS

- The stylesheet decides how .highlight looks
- JavaScript only decides who gets it

DOES NOT WORK

box.class = "card"

- class is a reserved word in JavaScript
- This sets a useless property and fails silently
- Use classList.add, or className

Making new elements

```
// 1. create it   (it exists, but is nowhere)
const li = document.createElement("li");

// 2. fill it
li.textContent = "Nusrat Jahan";
li.classList.add("student");

// 3. put it somewhere   <-- without this, nothing appears
const list = document.querySelector("#list");
list.appendChild(li);
```

1. create
2. fill
3. append

appendChild takes an element

- list.appendChild(li) works
- list.appendChild("Hello") throws: parameter 1 is not of type 'Node'

STEP 3 OR NOTHING

Created is not added

- Forget appendChild and the element exists only in memory
- No error. Nothing on the page. The most frustrating bug of the week

C1 · 15-30 MIN · LIVE BUILD

The pattern: data in, elements out

```
const students = [
  { name: "Rahim",  id: "C241001", marks: 78 },
  { name: "Karim",  id: "C241002", marks: 92 }
];

const list = document.querySelector("#student-list");

for (const s of students) {
  const li = document.createElement("li");       // create
  li.textContent = `${s.name} — ${s.marks}`;     // fill
  li.dataset.id = s.id;                          // remember where it came from
  if (s.marks >= 80) li.classList.add("top");    // decide its look
  list.appendChild(li);                          // append
}
```

This loop is the rest of the course

Products, comments, search results, API responses. Different words, identical shape.

Last class you had the array. This class you have the loop. That is a working page.

C1 · 15-30 MIN · LIVE BUILD

innerHTML or createElement?

```
// innerHTML: short
list.innerHTML = students
  .map(s => `<li>${s.name}</li>`)
  .join("");
```

```
// createElement: controlled
for (const s of students) {
  const li = document.createElement("li");
  li.textContent = s.name;
  list.appendChild(li);
}
```

innerHTML wins when

- You are writing the whole block at once
- The content is yours, not typed by a user
- You want fewer lines

createElement wins when

- The text came from a person
- You need dataset or classList on each item
- You are adding to a list, not replacing it

The trap with innerHTML

- list.innerHTML = ... replaces everything inside, every time. Use += and you rebuild the whole list on each pass.
- This course uses createElement by default. Use innerHTML when you can say why.

Live build: a student card board

board.html

Class Board

Rahim
78 marks

Karim TOP
92 marks

Nusrat
65 marks

Tasnim TOP
88 marks

4 students · 2 toppers · average 80.75

Every piece is from today

- querySelector finds the container
- createElement builds each card
- classList adds the top style
- dataset remembers the student id
- appendChild puts it on the page

The HTML is almost empty

- One container div and nothing inside it
- The CSS is written for you
- Every card you see was made by the script

Build it in four steps

1

Find the container

Log it. If it is null, your selector is wrong.

```
const board = document.querySelector("#board");
```

2

Loop and create

Nothing will appear yet. That is correct.

```
for (const s of students) { const card = document.createElement("div"); }
```

3

Fill and classify

Add the top class only when marks are 80 or more.

```
card.textContent = s.name;  card.classList.add("card");
```

4

Append

Now the page fills. Four cards, in order.

```
board.appendChild(card);
```

Mistakes you will make today

Cannot set properties of null

querySelector matched nothing. Check the # and the spelling.

parameter 1 is not of type 'Node'

You passed appendChild a string. It wants the element.

Nothing appears, no error

You created the element but never appended it.

Only the first one changed

querySelector gives one. You wanted querySelectorAll and a loop.

Tags showing as text

textContent where you wanted innerHTML.

My class did nothing

box.class = ... is silent. Use classList.add.

C1 · 48-50 MIN

Exit check

CONCEPTUAL

Answer in one sentence

- What is the difference between your HTML file and the DOM?
- Why does reloading undo everything your script did?

PRACTICAL

Show me

- Your board with four cards and two marked top
- One card inspected in the Elements tab

The three steps, out loud

create → fill → append. Miss the third and nothing happens, and nothing tells you.

Next class

C2 is practice: four exercises and six planted bugs. Bring today's board file. Nothing is clickable until C3.

WEEK 6 · CLASS 2

Practice: manipulating the DOM

Four exercises, six planted bugs, and a new debugging tool

C2 · 0-5 MIN

The plan for this hour

1	**Extend the board**	5-15 min	Add a count, a search field's value, and an image per card
2	**classList drills**	15-23 min	Mark, unmark and toggle, then read the result back
3	**Build from new data**	23-35 min	A different array, a different element type, your own code
4	**Six planted bugs**	35-46 min	Four of them are silent. The Elements tab is your tool

Last 4 minutes: assignment Part 1 check. Nothing today responds to a click. That is Class 3.

C2 · 5-23 MIN

Exercises 1 and 2

EXERCISE 1

Extend the board

- Show the real count in the heading, read from the array
- Give every card an image with a proper alt text
- Put the student id on each card with dataset
- Read one card's dataset back and log it

EXERCISE 2

classList drills

- Add .top to every student at 80 or above
- Remove it from one of them again
- toggle .compact on every card, twice, and explain the result
- Log classList.contains for one card before and after

```
// Exercise 1, the count. Why is this wrong?
document.querySelector("#count").textContent = "4 students";

// Exercise 2, the thing to notice
card.classList.toggle("compact");    // run it twice. What happened, and why?
```

A hardcoded 4 is wrong even when it is right. Add a student and find out why.

C2 · 23-35 MIN

Exercise 3: build it yourself

```
const books = [
  { title: "Eloquent JavaScript", author: "Haverbeke", year: 2018 },
  { title: "You Don't Know JS",   author: "Simpson",   year: 2020 },
  { title: "Refactoring UI",      author: "Wathan",    year: 2018 }
];

// Build one <article> per book inside #library
// Each one needs: an h3 with the title, a p with the author,
// a data-year attribute, and the class .old when year < 2019
```

The order that works

- Create the article, the h3 and the p
- Fill all three
- article.appendChild(h3), then article.appendChild(p)
- Finally library.appendChild(article)

RULE

No copying from C1

- Close board.js
- Write it from the three steps
- The handout is allowed, your old code is not

Nested elements

- An article holds an h3 and a p
- So you append twice: into the article, then the article into the page

C2 · 35-46 MIN

Exercise 4: six planted bugs

Three name themselves, but never all at once

You will see one error. Fix it and a different one appears. The page stays blank until the third is gone.

1	parameter 1 is not of type 'Node'	Console
2	Cannot read properties of null (appendChild)	Console
3	Cannot read properties of null (dataset)	Console
4	The author line is empty	Silent
5	Titles show their tags as text	Silent
6	Only one card turns amber	Silent

Bug 3 is the lesson of the week

- Its error is caused by a silent bug twenty lines earlier, in a different part of the file
- Where it hurts is not where it broke. Open the Elements tab and ask: does this element exist, and what classes does it carry?

C2 · 46-50 MIN

Assignment (5 marks)

PART 1 · 2 MARKS

In class, last 4 minutes

- Show your library built from the books array
- Show one article inspected in the Elements tab
- Say the three steps out loud

PART 2 · 3 MARKS

Home, before C3

- Fix all six planted bugs
- Add two more books and confirm the count updates itself
- Add a renderBooks(list) function that clears and rebuilds
- Commit and push to your Week 6 repository

Rubric

- Library renders from the array, nothing hardcoded — 1.5 · Classes and dataset correct — 1.0
- All six bugs fixed — 1.5 · renderBooks works and the page is pushed — 1.0

Why renderBooks matters

In Class 3 a click will call it. Writing it as a function now means Class 3 is one new line, not a rewrite.

C2 · 48-50 MIN

Exit check and what comes next

CONCEPTUAL

Answer in one sentence

- Why do most DOM bugs produce no Console error?
- What does appendChild need, and where does it put it?

PRACTICAL

Show me

- Three articles in the Elements tab
- One of them carrying the .old class

Class 3: events

- addEventListener, the event object, and preventDefault
- The page starts responding to the person using it
- Your renderBooks function gets called by a click

Today you changed the page. On Class 3 the page starts changing because of what someone does.